连接树(左深连接树、右深连接树、浓密树)

Improving Logical Query Plans
Optimization of Join Orders

连接顺序的优化

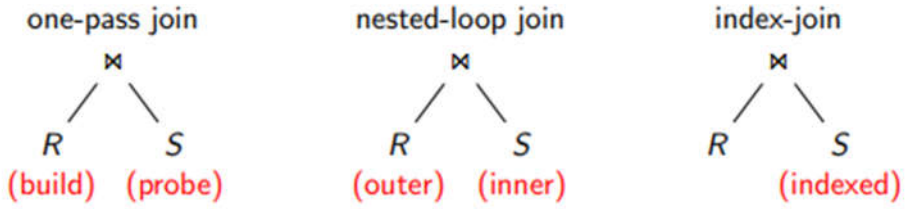
优化连接操作的执行顺序对于提高查询执行效率至关重要

- 连接操作的执行代价高
- 在数据库上执行几十个关系的连接很常见，如联机在线分析(Online Analytical Processing, OLAP)任务
- 不同连接顺序的执行代价差异巨大

连接关系的角色

尽管连接操作满足交换律，但在查询计划中连接操作的输入关系具有不同的作用

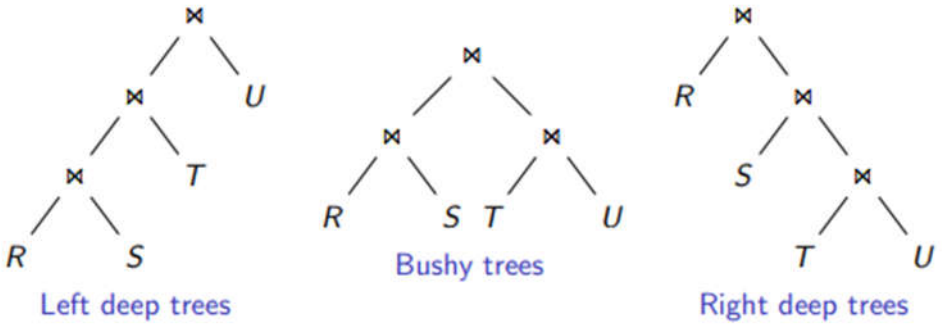
- 在$R \bowtie S$中，R是左关系(left relation)，S是右关系(right relation)
- 如果使用一趟连接(one-pass join)，则左关系是构建关系(build relation)，右关系是探测关系(probe relation)
- 如果使用嵌套循环连接(nested-loop join)，则左关系是外关系(outer relation)，右关系是内关系(inner relation)
- 如果使用索引连接(index-based join)，则右关系是有索引的关系(indexed relation)



连接树(Join Trees)

一组关系上的连接操作的执行顺序可以用连接树(join tree)来表示

- 左深连接树(left-deep join tree): 只有一个关系是左关系(left relation)，其他关系都是右关系(right relation)
- 右深连接树(right-deep join tree): 只有一个关系是右关系，其他关系都是左关系(left relation)
- 浓密树(bushy tree): 除左深连接树和右深连接树以外的其他连接树



左深连接树(Left-Deep Join Trees)

System R的查询优化器只考虑左深连接树

优点1: 在给定关系上，全部左深连接树的数量比全部连接树少得多

- $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_n$的全部左深连接树的数量为$n!$
- $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_n$的全部连接树的数量为$n!C_n$，其中$C_n$表示含有$n$个叶子节点的树形的数量(Catalan数)

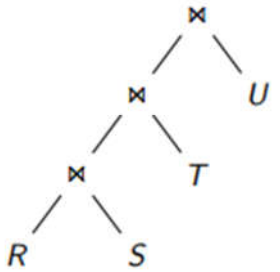
$$C_1 = 1,$$
$$C_n = \sum_{i=1}^{n-1} C_i C_{n-i} \qquad \text{if } n > 1$$

优点2: 基于左深连接树的查询计划通常比基于其他类型连接树的查询计划的执行效率更高

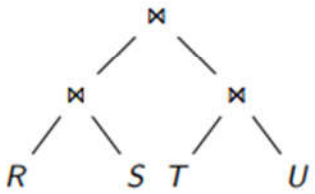
- 可能形成完全通畅的流水线(fully piplined plans)

为什么左深连接树查询计划更高效?

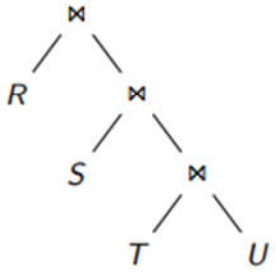
如果下列查询计划全部使用一趟连接(one-pass join)算法，那么在流水线查询执行模型(piplining model)下，左深连接树查询计划在任何时刻对内存的需求都比其他查询计划更低



At any time, only one of R, $R \bowtie S$, and $R \bowtie S \bowtie T$ will be held in the buffers



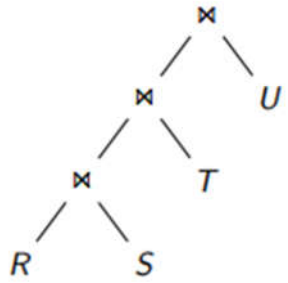
$R \bowtie S$ and T must be held in the buffers at the same time



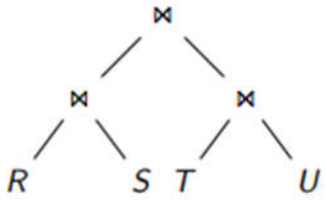
All of R, S, and T must be held in the buffers at the same time

为什么左深连接树查询计划更高效?

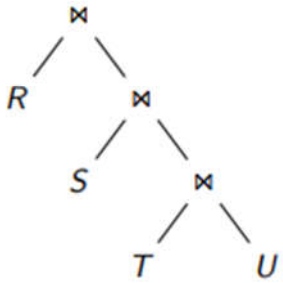
如果下列查询计划全部使用嵌套循环连接(nested-loop join)算法，那么在流水线查询执行模型(piplining model)下，左深连接树查询计划不需要多次构建中间关系



Both $R \bowtie S$ and $R \bowtie S \bowtie T$ are constructed only once



$R \bowtie S$ is constructed only once, but $T \bowtie U$ needs to be constructed more than once (if not materialized)



$S \bowtie T \bowtie U$ needs to be constructed more than once, so does $T \bowtie U$

优化连接顺序的动态规划方法

Example (连接顺序优化)

Choose the best join order for $R(a,b) \bowtie S(b,c) \bowtie T(c,d) \bowtie U(d,a)$

$R(a,b)$	$S(b,c)$	$T(c,d)$	$U(d,a)$
$T(R) = 1000$	$T(S) = 1000$	$T(T) = 1000$	$T(U) = 1000$
$V(R,a) = 100$			$V(U,a) = 50$
$V(R,b) = 200$	$V(S,b) = 100$		
	$V(S,c) = 500$	$V(T,c) = 20$	
		$V(T,d) = 50$	$V(U,d) = 1000$

Best orders of joins on 2 relations

	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

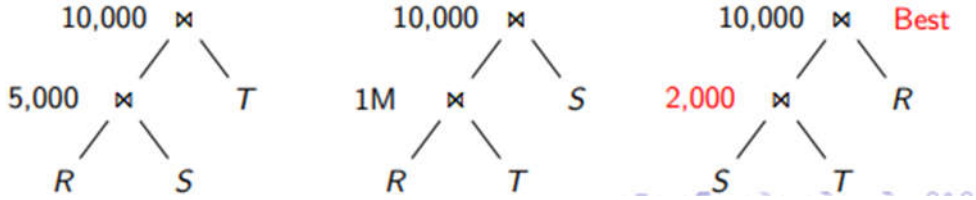
Best orders of joins on 2 relations

	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$

Best orders of joins on 3 relations

	$R \bowtie S \bowtie T$	$R \bowtie S \bowtie U$	$R \bowtie T \bowtie U$	$S \bowtie T \bowtie U$
Est. size	10,000	50,000	10,000	2,000
Est. cost	2,000	5,000	1,000	1,000
Best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$

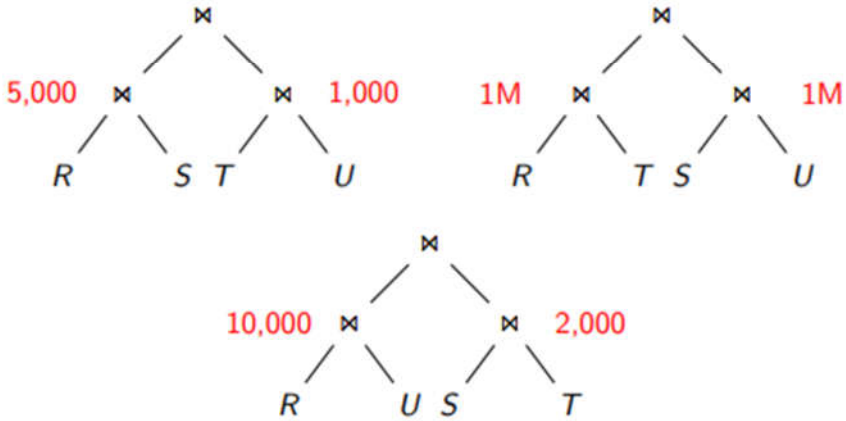
For example, the best plan for $R \bowtie S \bowtie T$ is obtained as follows



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 1)

Best orders of joins on 2 relations

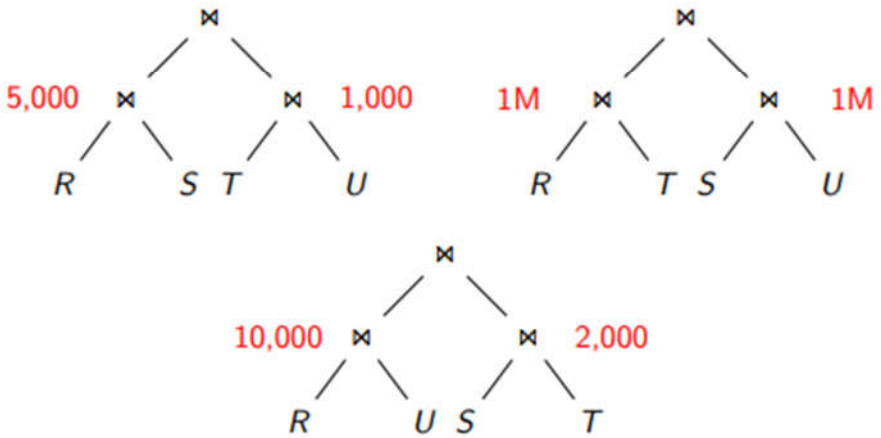
	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 1)

Best orders of joins on 2 relations

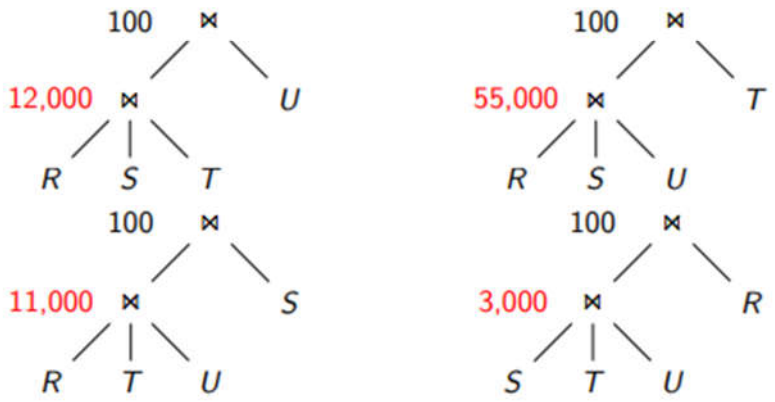
	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$
Est. size	5,000	1M	10,000	2,000	1M	1,000
Est. cost	0	0	0	0	0	0
Best plan	$R \bowtie S$	$R \bowtie T$	$R \bowtie U$	$S \bowtie T$	$S \bowtie U$	$T \bowtie U$



The best plan for $R \bowtie S \bowtie T \bowtie U$ is obtained as follows (phase 2)

Best orders of joins on 3 relations

	$R \bowtie S \bowtie T$	$R \bowtie S \bowtie U$	$R \bowtie T \bowtie U$	$S \bowtie T \bowtie U$
Est. size	10,000	50,000	10,000	2,000
Est. cost	2,000	5,000	1,000	1,000
Best plan	$(S \bowtie T) \bowtie R$	$(R \bowtie S) \bowtie U$	$(T \bowtie U) \bowtie R$	$(T \bowtie U) \bowtie S$



查询执行模型：物化、流水线

查询计划的执行方法

如何执行由多个操作构成的查询计划(query plan)?

- 方法1: 物化执行(Materialization)
- 方法2: 流水线执行(Piplining)

物化执行(Materialization)

- 自底向上执行查询计划中的操作
- 每个中间操作的执行结果写入临时关系文件，作为后续操作的输入

Example (物化执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - - → (输出关系文件)

|

$\sigma_{salary<75000}$; use index ← - - - - - - → (临时关系文件)

|

instructor ← - - - - - - - - - - - - (输入关系文件)

物化执行的缺点

缺点1: 物化(materialize)临时关系增加了查询执行的代价

- 执行完一个操作后，临时关系必须写入文件(除非临时关系非常小)
- 执行后续操作时，临时关系文件再被读入缓冲区

缺点2: 用户获得查询结果的时间延迟大

Example (物化执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - → (Output Relation)

|

$\sigma_{salary<75000}$; use index ← - - - - → (Temprory Relation)

|

instructor ← - - - - - - - - - - - - (Input Relation)

流水线执行(Piplining)

将查询计划中若干操作组成流水线(pipline)，一个操作的结果直接传给流水线中下一个操作

- 避免产生一些临时关系，避免了读写这些临时关系文件的I/O开销
- 用户能够更快地得到查询结果

几乎所有DBMS都使用流水线执行查询计划

Example (流水线执行)

$\Pi_{ID,salary}$ - - - - - - - - - - - → (Output Relation)

| piplined

$\sigma_{salary<75000}$; use index (~~Temporary Relation~~)

|

instructor ← - - - - - - - - - - - - (Input Relation)